

Solving Problems by Searching

*Chapter 3 of Artificial Intelligence: A Modern Approach, 4th Edition
Russell & Norvig*

Lecture Notes

Search: Uninformed and Informed Strategies

Roadmap

Problem-Solving Agents

Example Problems

Search Algorithms

Uninformed Search Strategies

Informed (Heuristic) Search

Heuristic Functions

Chapter Summary

From Percepts to Plans: Why Search?

- When the correct action is **not immediately obvious**, an agent must **plan ahead**.
- It considers a **sequence** of actions forming a path to a goal state.
- This computational process is called **search**, and an agent that does it is a **problem-solving agent**.
- Problem-solving agents use **atomic representations**: states are indivisible black boxes, with no internal structure visible to the algorithm.

Scope of this chapter

Simplest environments only:

- episodic
- single-agent
- fully observable
- deterministic
- static
- discrete
- known

We distinguish **informed** algorithms (agent can estimate distance to goal) from **uninformed** algorithms (no such estimate).

Motivating Example: A Touring Vacation in Romania

- Agent is enjoying a touring vacation in Romania: sights, nightlife, avoiding hangovers. . .
- Currently in the city of **Arad**, with a **nonrefundable flight out of Bucharest** tomorrow.
- Three roads leave Arad: to Sibiu, Timisoara, Zerind. None is the goal — unless the agent knows Romanian geography, it doesn't know which to take.

If the agent has no information (environment is unknown)

It can do no better than executing an action at random.

This chapter's assumption

The agent **always has access to information about the world** (e.g., a road map), so it can follow a structured problem-solving process.

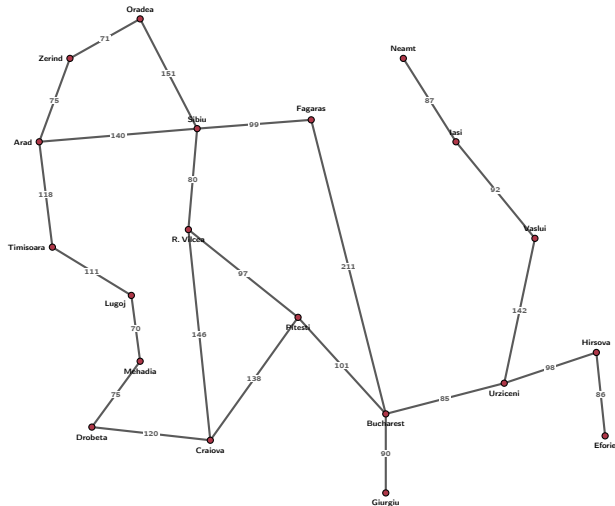
The Four-Phase Problem-Solving Process

1. **Goal formulation:** The agent adopts a goal (e.g., reach Bucharest). Goals organize behavior by limiting the objectives, and hence the actions to consider.
2. **Problem formulation:** The agent devises a description of the states and actions necessary to reach the goal — an *abstract model* of the relevant part of the world.
3. **Search:** Before acting in the real world, the agent *simulates* sequences of actions in its model, until it finds one that reaches the goal (a **solution**), or concludes none exists.
4. **Execution:** The agent executes the actions in the solution, one at a time.

Open-loop vs. closed-loop execution

In a fully observable, deterministic, known environment, the solution to a problem is a *fixed sequence of actions*. If the model is correct, the agent can execute it **open-loop** (ignoring percepts). If the environment might be wrong, or is nondeterministic, a **closed-loop** approach that monitors percepts is safer (Section 4.4).

The Romania Road Map



Running Example: A simplified road map of part of Romania, with road distances in miles.

Search Problem: Formal Definition

A search **problem** can be defined formally by five components:

- A set of possible **states** the environment can be in — the **state space**.
- The **initial state** the agent starts in, e.g. *Arad*.
- A set of **goal states** (sometimes one, sometimes a set, sometimes a property tested with an Is-Goal method).
- The **actions** available: $\text{Actions}(s)$ returns the finite set of actions applicable in state s .

$$\text{Actions}(\text{Arad}) = \{ \text{ToSibiu}, \text{ToTimisoara}, \text{ToZerind} \}$$

- A **transition model**: $\text{Result}(s, a)$ returns the state resulting from doing a in s .

$$\text{Result}(\text{Arad}, \text{ToZerind}) = \text{Zerind}$$

- An **action cost function**, $\text{Action-Cost}(s, a, s')$ or $c(s, a, s')$, giving the numeric cost of applying action a in state s to reach s' .

Paths, Solutions, and the State Space as a Graph

- A **path** is a sequence of actions; a **solution** is a path from the initial state to a goal state.
- We assume action costs are **additive**: total path cost = sum of individual action costs.
- An **optimal solution** has the lowest path cost among all solutions.
- The state space can be represented as a **graph**: vertices are states, directed edges are actions.
 - The Romania map is such a graph; each road links two cities with two actions (one each direction).

Technical caveats (footnotes)

- If $\text{Actions}(s)$ could be infinite, we'd need techniques beyond this chapter.
- With a cycle of *net negative* cost, the optimal solution would traverse it forever — we assume all action costs are positive.
- Zero-cost actions are fine as long as consecutive zero-cost actions are bounded (else require costs $\geq \epsilon > 0$).

Abstraction: Choosing the Right Level of Detail

- The problem formulation of “get to Bucharest” is a **model** — an abstract mathematical description, not the real thing.
- Compare the atomic state “*Arad*” to an actual cross-country trip: companions, radio, scenery, traffic, weather. . . all irrelevant *to this problem* and left out.
- Removing detail from a representation is called **abstraction**.

Validity and usefulness of an abstraction

- An abstraction is **valid** if any abstract solution can be elaborated into a solution in the more detailed world (e.g., every detailed state “in Arad” has a detailed path to some state “in Sibiu”).
- An abstraction is **useful** if carrying out each abstract action is *easier* than the original problem (e.g., “drive from Arad to Sibiu” needs no further planning by an average driver).

Were it not for the ability to construct useful abstractions, (intelligent) agents would be completely swamped by the real world.

Standardized vs. Real-World Problems

Standardized problem

Intended to illustrate or exercise problem-solving methods. Concise, exact description $\Rightarrow$ useful as a benchmark for comparing algorithms.

We will formally specify several standardized problems (grid world, vacuum world, sliding-tile puzzles, Knuth's problem), then survey real-world problems.

Real-world problem

Solutions are actually used by people; formulation is idiosyncratic, not standardized (e.g., every robot has different sensors).

Grid World

- A two-dimensional rectangular array of square cells; agents move cell to cell (horizontally/vertically, sometimes diagonally).
- Cells may contain objects that an agent can pick up, push, or act upon; obstacles block movement.

Sokoban puzzle

Agent pushes boxes onto storage locations. At most one box per cell; pushing forward into a box moves both agent and box forward if the far cell is empty. Cannot push into another box or a wall.

For n non-obstacle cells and b boxes: $\frac{n \times n!}{b! (n - b)!}$ states — e.g., an 8×8 grid with a dozen boxes has over 200 *trillion* states.

Formalizing the Vacuum World (as a Grid Problem)

- **States:** which objects (agent, dirt) are in which cells. Two-cell version: $2 \cdot 2 \cdot 2 = 8$ states. General n -cell world: $n \cdot 2^n$ states.
- **Initial state:** any state can be designated as initial.
- **Actions:** two-cell world uses $\{Suck, Left, Right\}$. Multi-cell 2D worlds need more: **absolute** moves (*Upward, Downward*, plus *Left/Right*), or **egocentric** actions (*Forward, Backward, TurnRight, TurnLeft*).
- **Transition model:** *Suck* removes dirt from the agent's cell; *Forward* moves ahead one cell unless blocked by a wall (no effect); *Backward* is the opposite; turns change orientation by 90° .
- **Goal states:** every cell is clean.
- **Action cost:** each action costs 1.

8 states, 3 actions each:



Sliding-Tile Puzzles: The 8-Puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

A 3×3 grid, 8 numbered tiles, one blank; the **15-puzzle** uses a 4×4 grid. Standard 8-puzzle formulation:

- **States:** location of each tile. **Actions:** blank moves *Left/Right/Up/Down* (not all applicable at edges/corners).
- **Transition model:** swaps blank with the adjacent tile. **Goal:** numbers in order. **Action cost:** 1 each.

Knuth's¹ Infinite State-Space Puzzle

- Devised by Donald Knuth (1964); illustrates how **infinite state spaces** can arise.
- Starting with the number 4, a sequence of $\sqrt{\cdot}$, $\lfloor \cdot \rfloor$, and factorial operations can reach any desired positive integer.

$$\left\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}}}} \right\rfloor = 5$$

States: positive reals.

Initial state: 4.

Actions: apply $\sqrt{\cdot}$, $\lfloor \cdot \rfloor$, or factorial (integers only).

Transition model: math definitions.

Goal: desired integer.

Action cost: 1 each.

The shortest path to 5 goes through $(4!)! = 620,448,401,733,239,439,360,000$. Infinite state spaces arise often in tasks generating math expressions, circuits, proofs, programs.

¹https://en.wikipedia.org/wiki/Donald_Knuth

Real-World Problems (I): Route-Finding & Touring

Route-finding problem

Specified locations and transitions along edges (Web sites, in-car GPS). Main complications: traffic-dependent costs, rerouting due to closures.

Touring problems

Visit a set of locations rather than reach one goal.

- **Traveling Salesperson Problem (TSP)**: visit every city with cost $< C$ (or minimal cost). Huge research effort; extends to vehicle fleets — e.g., a Boston school-bus routing system saved \$5M and cut pollution (Bertsimas *et al.*, 2019).

Airline travel-planning

States include location *and* current time (plus fare-basis history). **Actions**: take a flight from current location after current time. **Goal**: destination city (possibly “arrive on a nonstop flight”). **Cost**: money, wait time, flight time, customs, seat quality, time of day, plane type, frequent-flyer rewards. . .

Real-World Problems (II): Layout, Navigation, Assembly

- **VLSI layout:** position millions of components/connections on a chip to minimize area, delay, stray capacitance, maximize yield. Split into **cell layout** and **channel routing**.
- **Robot navigation:** generalizes route-finding; a circular robot on a flat surface has a 2D search space; robots with arms/legs need one dimension per joint angle (Ch. 26). Must also handle sensor/motor error, partial observability, other agents.
- **Automatic assembly sequencing:** find a feasible (then optimized) order to assemble parts of an object (e.g., an electric motor) — standard industry practice since the 1970s. Wrong order $\Rightarrow$ may be impossible to add a later part without undoing work. Generating legal actions is the expensive part.
- **Protein design:** find a sequence of amino acids that folds into a 3D protein with properties that could cure a disease.

Search Trees vs. the State Space

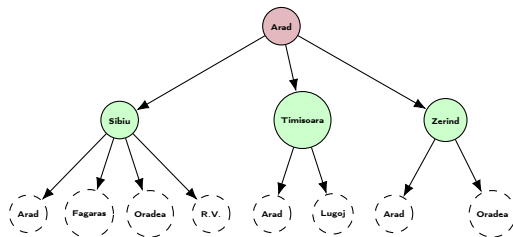
- A **search algorithm** takes a problem as input and returns a solution or an indication of failure.
- It superimposes a **search tree** over the state-space graph: paths from the initial state, trying to reach a goal.
- Each **node** in the search tree corresponds to a state; edges correspond to actions. The root corresponds to the initial state.

Key distinction

The **state space** describes the (possibly infinite) set of states and the transitions between them — it is fixed by the problem. The **search tree** describes paths between these states toward the goal, and *depends on the algorithm*: it may contain multiple nodes for the same state (multiple paths to it), while each tree node has a unique path back to the root.

We **expand** a node by considering the available Actions for its state, using Result to **generate child nodes** (each with the expanded node as **parent**).

Search Tree Growth on the Romania Example



- After expanding **Arad** (lavender) we reach Sibiu, Timisoara, Zerind (green = **frontier**: generated but not yet expanded).
- Dashed nodes: states that *could* be generated next.
- Suppose we expand Sibiu next: we get 6 unexpanded frontier nodes. Note a cycle Arad→Sibiu→Arad is possible but can't be part of an optimal path.

Frontier, Reached, and the Separation Property

- Any state that has had a node generated for it has been **reached** (whether expanded or not).
- The **frontier** = set of *reached but not yet expanded* nodes (some authors call this the *open list*; the *closed list* = reached minus frontier).

Separation property

The frontier separates the state-space graph into two regions: an **interior** where every state has been expanded, and an **exterior** of states not yet reached. Every path from the initial state to an unreached state must pass through a frontier state.

Think of breadth-first search's frontier as the surface of an ever-expanding sphere; depth-first search's frontier is just a radius of that sphere.

Best-First Search: The General Algorithm

```

1: function Best-First-Search(problem, f)
2:   node  $\leftarrow$  Node(State = problem.Initial)
3:   frontier  $\leftarrow$  a priority queue ordered by f, with node as an element
4:   reached  $\leftarrow$  a lookup table, with one entry {problem.Initial  $\mapsto$  node}
5:   while not Is-Empty(frontier) do
6:     node  $\leftarrow$  Pop(frontier)
7:     if problem.Is-Goal(node.State) then return node
8:     end if
9:     for each child in Expand(problem, node) do
10:      s  $\leftarrow$  child.State
11:      if s not in reached or child.Path-Cost < reached[s].Path-Cost then
12:        reached[s]  $\leftarrow$  child; add child to frontier
13:      end if
14:    end for
15:  end while
16:  return failure
17: end function
    
```

A very general template: choosing a node with minimum $f(n)$, the **evaluation function**, gives different algorithms for different f . On each iteration we choose a frontier node with minimum $f(n)$; return it if its state is a goal, otherwise Expand it to generate children.

Expand: Generating Successor Nodes

```

1: function Expand(problem, node)                                     ▷ yields nodes
2:   s ← node.State
3:   for each action in problem.Actions(s) do
4:     s' ← problem.Result(s, action)
5:     cost ← node.Path-Cost + problem.Action-Cost(s, action, s')
6:     yield Node(State= s', Parent= node, Action= action, Path-Cost= cost)
7:   end for
8: end function

```

- For each state *s* reachable from *node* by one action, Expand builds a new child node: its path cost is the parent's path cost plus the cost of the new action.
- Each child records its Parent and the Action taken, so a solution path can be recovered by walking parent pointers back to the root.
- “**yield**” means Expand produces child nodes one at a time (a generator), rather than building the whole list at once — see Appendix B.

Search Node Data Structure

A node in the search tree stores four components:

- `node.State`: the state the node corresponds to.
- `node.Parent`: the node in the tree that generated this node.
- `node.Action`: the action applied to the parent's state to generate this node.
- `node.Path-Cost`: total cost of the path from the initial state to this node ($g(node)$).

Following Parent pointers back from a node recovers the states and actions along the path — from a goal node, this recovers the **solution**.

The frontier needs a queue

- **Priority queue**: pops minimum-cost node by f — used in best-first search.
- **FIFO queue**: pops the oldest node — used in breadth-first search.
- **LIFO queue** (stack): pops the newest node — used in depth-first search.

reached is usually a hash table mapping states to nodes.

Redundant Paths and Cycles

- A **repeated state**, generated by a **cycle** (loopy path), makes the complete search tree *infinite* even though the state space is finite (e.g., 20 states in Romania, but no limit on how often a loop can be traversed).
- A **redundant path** is any worse way of reaching a state already reached (a cycle is a special case).

The cost of ignoring redundancy

A 10×10 grid world, 8-connected. In 9 moves or fewer the agent reaches any of the 100 squares — but there are almost $8^9 > 100$ million paths of length 9! Eliminating redundant paths can make search *roughly a million times faster*.

“Algorithms that cannot remember the past are doomed to repeat it.”

Three Approaches to Redundant Paths

1. **Remember all reached states** (as Best-First-Search does): detects all redundant paths, keeps only the best path to each state. Best when the state space has many redundant paths and *reached* fits in memory.
2. **Don't worry about it**: for problem formulations where two paths rarely/never reach the same state (e.g., assembly where parts are added in a strict order), we can save memory by not tracking *reached* at all.
3. **Compromise**: check for *cycles only* (not all redundant paths) by following the chain of parent pointers — some implementations check all ancestors (eliminates all cycles); others check only a few links (constant time, catches short cycles).

Terminology

A **graph search** algorithm checks for redundant paths; a **tree-like search**² does not. Best-First-Search is a graph search; removing all references to *reached* yields a tree-like search — less memory, but may examine redundant paths and run slower.

²"Tree-like" because the state space is still the same graph; we just treat it *as if* it were a tree.

Measuring Problem-Solving Performance

Four criteria for evaluating a search algorithm:

- **Completeness:** Is it guaranteed to find a solution when one exists, and correctly report failure otherwise?
- **Cost optimality:** Does it find a solution with the lowest path cost of all solutions?
- **Time complexity:** How long to find a solution (seconds, or states/actions considered)?
- **Space complexity:** How much memory is needed?

Completeness needs systematicity

To be complete in an infinite state space, a search algorithm must be **systematic** in how it explores — e.g., a spiral path on an infinite grid that covers all cells s steps away before cells $s+1$ steps away. In an infinite space *with no solution*, a sound algorithm must search forever (it can't know the next state won't be the goal).

Complexity Measures: $|V|+|E|$ vs. b, d, m

- For an **explicit** graph (e.g., the Romania map), the natural measure is $|V| + |E|$ (vertices + edges).
- For an **implicit** state space (defined only by initial state + actions + transition model), we instead use:

b

branching factor: max
number of successors of any
node

d

depth: number of actions in
an optimal solution

m

maximum number of actions in
any path

Uninformed Search: No Clue How Close We Are

- An **uninformed** search algorithm is given no clue about how close a state is to the goal(s) — it only knows the problem definition.
- Example: an agent in Arad with no knowledge of Romanian geography has no clue whether Zerind or Sibiu is the better first step.
- Contrast with **informed** search (Section 3.5), which uses a heuristic function.

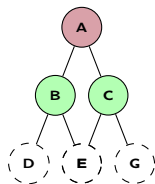
Five uninformed strategies covered

Breadth-first search, uniform-cost search (Dijkstra)³, depth-first search, depth-limited & iterative-deepening search, bidirectional search.

³https://en.wikipedia.org/wiki/Edsger_W._Dijkstra

Breadth-First Search

- When all actions have the **same cost**, an appropriate strategy: expand the root first, then *all* successors of the root, then *their* successors, and so on.
- Equivalent to Best-First-Search with $f(n) = \text{depth of } n$.
- Efficiency tricks:** use a FIFO queue (new/deeper nodes go to the back); track *reached* as a set of states (not state $\rightarrow$ node map, since once reached, no better path can be found); do an **early goal test** (check as soon as a node is *generated*) rather than the **late goal test** best-first search uses (when *popped*).



Node to expand next marked by frontier color; BFS goes level by level.

Breadth-First Search: Algorithm

```

1: function Breadth-First-Search(problem)
2:   node  $\leftarrow$  Node(problem.Initial)
3:   if problem.Is-Goal(node.State) then return node
4:   end if
5:   frontier  $\leftarrow$  a FIFO queue, with node as an element
6:   reached  $\leftarrow$  {problem.Initial}
7:   while not Is-Empty(frontier) do
8:     node  $\leftarrow$  Pop(frontier)
9:     for each child in Expand(problem, node) do
10:      s  $\leftarrow$  child.State
11:      if problem.Is-Goal(s) then return child
12:      end if
13:      if s not in reached then add s to reached; add child to frontier
14:      end if
15:    end for
16:  end while
17:  return failure
    
```

Breadth-First Search: Complexity and Properties

Note the **early goal test** in the algorithm (line 11, checked when a child is *generated*) versus the *late* goal test of generic best-first search (checked only when a node is *popped*) — this lets BFS avoid ever adding a goal node's siblings to the frontier unnecessarily.

Complexity (b = branching factor, solution at depth d)

The root generates b nodes, each generates b more (b^2 at level 2), and so on. Total nodes generated:

$$1 + b + b^2 + \dots + b^d = O(b^d)$$

Both time and space are $O(b^d)$ — all nodes remain in memory.

Exponential bounds are scary

$b=10$, 1M nodes/sec, 1KB/node $\Rightarrow d=10$ takes <3 hrs but needs **10 terabytes**; $d=14$ needs **3.5 years** even with infinite memory. *Memory is the bigger problem for BFS than execution time.*

- **Complete** if b is finite.
- **Cost-optimal** *only* when all action costs are equal (otherwise it may return a shallower but costlier solution).

Uniform-Cost Search (Dijkstra's Algorithm)

- When actions have **different costs**, use best-first search with $f(n) = g(n)$, the path cost from root to n .
- Known as **Dijkstra's algorithm** in theoretical CS, **uniform-cost search** in AI.
- BFS spreads out in waves of uniform *depth*; UCS spreads out in waves of uniform *path-cost*.
- Implementation: $\text{Uniform-Cost-Search}(\text{problem}) \equiv \text{Best-First-Search}(\text{problem}, \text{Path-Cost})$.

Sibiu → Bucharest

Successors of Sibiu: R. Vilcea (80), Fagaras (99). Expand R. Vilcea → Pitesti at $80+97=177$. Expand Fagaras (next-lowest) → Bucharest at $99+211=310$ (a goal, but *not yet returned* — tested at expansion, not generation!). Expand Pitesti → second path to Bucharest at $80+97+101 = 278$, replacing the 310-cost entry. Bucharest (278) is now cheapest, so it's expanded and returned.

Complexity: $O(b^{1+\lceil C^*/\epsilon \rceil})$ (C^* = optimal cost, ϵ = min action cost) — can be far worse than b^d .

Complete and cost-optimal (all costs > 0).

Depth-First Search

- Always expands the **deepest** node in the frontier first — $f(n) = -\text{depth}(n)$ in best-first terms.
- Usually implemented as a **tree-like** search (LIFO queue / stack), *not* keeping a *reached* table.
- Search proceeds immediately to the deepest level, then “backs up” to the next node with unexpanded successors.

Properties: mostly bad news

- **Not cost-optimal:** returns the *first* solution found, not necessarily cheapest.
- Finite tree-shaped spaces: efficient and complete.
- Finite *acyclic* graphs: may expand the same state many times via different paths, but eventually covers the whole space.
- **Cyclic** spaces: can loop forever (some implementations check each new node against its ancestors for cycles).
- **Infinite** spaces: *not systematic* $\Rightarrow$ **incomplete**, even with no cycles.

So why use it? Memory

Backtracking Search

A variant of DFS using *even less* memory:

- Only **one** successor is generated at a time (not all); each partially expanded node remembers which successor to generate next.
- Successors are generated by **modifying** the current state description directly, rather than allocating a new state.
- Memory: just *one* state description + a path of $O(m)$ actions — a big savings over $O(bm)$ states.
- We can maintain an efficient set of the states on the current path, checking for a cyclic path in $O(1)$ rather than $O(m)$.
- Requires the ability to **undo** each action when backtracking.

Backtracking is critical to the success of many problems with large state descriptions, such as robotic assembly. (More in Chapter 5.)

Depth-Limited Search

- To keep DFS from wandering down an infinite path, supply a **depth limit** ℓ ; nodes at depth ℓ are treated as if they had no successors.
- Time complexity $O(b^\ell)$; space complexity $O(b\ell)$.
- A poor choice of ℓ makes the algorithm **incomplete** again.
- We can still eliminate cycles cheaply by checking a few ancestor links (longer cycles are handled by the depth limit itself).

Choosing ℓ : the *diameter*

E.g., Romania has 20 cities, so $\ell=19$ is *valid*, but studying the map shows any city reaches any other in ≤ 9 actions — the **diameter** of the state-space graph gives a much better (smaller) limit. For most problems, though, we won't know a good limit in advance.

Iterative Deepening Search: Idea

- Solves the problem of picking ℓ : try **all** values $\ell = 0, 1, 2, \dots$ until a solution is found, or depth-limited search returns *failure* (not just *cutoff*).
- Combines benefits of DFS (memory) and BFS (completeness/optimality for unit costs).
- Each iteration re-does the work of all previous iterations, then goes one level deeper — see the next slide for why this is not as wasteful as it sounds.

Iterative deepening runs Depth-Limited-Search with limit $\ell = 0$, then $\ell = 1$, then $\ell = 2, \dots$. Each run explores the *same* shallow nodes again, but only the deepest run does any genuinely new work.

Iterative Deepening Search: Algorithm

```

1: function Iterative-Deepening-Search(problem)
2:   for depth = 0 to  $\infty$  do
3:     result  $\leftarrow$  Depth-Limited-Search(problem, depth)
4:     if result  $\neq$  cutoff then return result
5:   end if
6: end for
7: end function

8: function Depth-Limited-Search(problem,  $\ell$ )
9:   frontier  $\leftarrow$  a LIFO queue (stack) with Node(problem.Initial)
10:  result  $\leftarrow$  failure
11:  while not Is-Empty(frontier) do
12:    node  $\leftarrow$  Pop(frontier)
13:    if problem.Is-Goal(node.State) then return node
14:    end if
15:    if Depth(node) >  $\ell$  then result  $\leftarrow$  cutoff
16:    else if not Is-Cycle(node) then
17:      for each child in Expand(problem, node) do add child to frontier
18:    end for
19:    end if
20:  end while
21:  return result
    
```

Iterative Deepening: Complexity

- Time: $O(b^d)$ with a solution, $O(b^m)$ with none.
- Memory: $O(bd)$ with a solution, $O(bm)$ on finite spaces with none — **modest**, like DFS.
- Complete on finite acyclic state spaces (like BFS); optimal when all costs equal.

Isn't re-generating shallow nodes wasteful?

Nodes at the bottom level (d) generated once; next-to-bottom generated twice; ...; children of root generated d times:

$$N(\text{IDS}) = d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$$

Asymptotically the same as BFS! Numeric example ($b=10, d=5$): $N(\text{IDS}) = 123,450$ vs. $N(\text{BFS}) = 111,110$ — close.

A hybrid approach: run BFS until memory is nearly full, then run iterative deepening from every frontier node. **Iterative deepening is the preferred uninformed method when the space is larger than fits in memory and the solution depth is unknown.**

Bidirectional Search

- Simultaneously searches **forward** from the initial state and **backward** from the goal state(s), hoping the two searches meet.
- Motivation: $b^{d/2} + b^{d/2} \ll b^d$ (e.g., $50,000\times$ less when $b=d=10$).
- Requires reasoning backwards: if s' is a successor of s forward, then s is a successor of s' backward.

Bidirectional best-first search (BIBF-Search) keeps two frontiers $frontier_F$, $frontier_B$ and two reached tables $reached_F$, $reached_B$. At each step, expand the frontier (F or B) whose top has lower f -value. When a child is also present in the *other* reached table, join the two partial paths (Join-Nodes) to form a candidate solution.

Careful: not automatically optimal

The first solution found need not be the cheapest — Terminated must prove no better solution remains before stopping. With $f = g$ (bidirectional *uniform-cost*), no node with cost $> C^*/2$ is ever expanded — a considerable speedup.

Comparing Uninformed Search Algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(bd)$	$O(b^{d/2})$

¹ complete if b is finite and the state space has a solution or is finite.

² complete if all action costs are $\geq \epsilon > 0$.

³ cost-optimal if action costs are all identical.

⁴ if both directions are breadth-first or uniform-cost.

(Table for *tree-like* search; for graph searches that check repeated states, DFS becomes complete on finite spaces, and space/time are bounded by $|V| + |E|$.)

Informed Search and the Heuristic Function

- An **informed search** strategy uses domain-specific hints about the location of goals to find solutions more efficiently.
- The hints come from a **heuristic function** $h(n)$:

$h(n)$ = estimated cost of the cheapest path from the state at n to a goal

- Example: in route-finding, estimate remaining distance via the **straight-line distance** on the map.

City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Fagaras	176	Pitesti	100
Rimnicu Vilcea	193	Sibiu	253
Timisoara	329	Zerind	374

(Straight-line distances to Bucharest, h_{SLD} — Fig. 3.16, abridged.)

Greedy Best-First Search

- Expands first the node with lowest $h(n)$ — the node that *appears* closest to the goal: $f(n) = h(n)$.
- Romania example: from Arad, h_{SLD} says Sibiu (253) is closer than Zerind (374) or Timisoara (329) → expand Sibiu. Then Fagaras (176) is closest → expand it. Fagaras generates Bucharest, the goal!

Greedy, but not optimal

The path found, Arad→Sibiu→Fagaras→Bucharest, is **32 miles longer** than the path via R. Vilcea and Pitesti. Greedy best-first search tries to get as close to the goal as possible on *each* step — but greediness can lead to worse overall results.

Complete in finite spaces, not in infinite ones. Worst-case time/space $O(|V|)$; with a good heuristic, complexity can drop substantially, sometimes to $O(bm)$.

A Search*

- The most common informed algorithm: best-first search with

$$f(n) = g(n) + h(n)$$

where $g(n)$ = path cost so far, $h(n)$ = estimated cost to a goal, so $f(n)$ = estimated cost of the best path continuing through n to a goal.

- Romania trace: Bucharest first appears on the frontier at step (e) with $f=450$, but is *not* selected — Pitesti has $f=417$, lower. Only at step (f), when a *second* path to Bucharest scores $f=418$ (still less than any alternative), is it expanded and returned as optimal.

Admissible heuristic

$h(n)$ is **admissible** if it *never overestimates* the true cost to reach a goal (hence *optimistic*).

h_{SLD} is admissible: straight-line distance never exceeds road distance.

A* is **complete**. Whether it's **cost-optimal** depends on properties of h .

A Cost-Optimality: Proof Sketch*

Claim: with an admissible h , A^* is cost-optimal. *Proof by contradiction:*

- Suppose the optimal path has cost C^* , but A^* returns a path with cost $C > C^*$.
- Then some node n on the optimal path must be **unexpanded** (else the optimal solution would have been returned).

$$f(n) > C^* \quad (\text{otherwise } n \text{ would have been expanded})$$

$$f(n) = g(n) + h(n) \quad (\text{by definition})$$

$$f(n) = g^*(n) + h(n) \quad (n \text{ is on an optimal path})$$

$$f(n) \leq g^*(n) + h^*(n) \quad (\text{admissibility: } h(n) \leq h^*(n))$$

$$f(n) \leq C^* \quad (\text{by definition, } C^* = g^*(n) + h^*(n))$$

Contradiction

The first and last lines contradict each other ($f(n) > C^*$ and $f(n) \leq C^*$) $\Rightarrow A^*$ can only return cost-optimal paths. ■

Consistency: A Stronger Property

Consistent heuristic

$h(n)$ is **consistent** if, for every node n and every successor n' generated by action a :

$$h(n) \leq c(n, a, n') + h(n')$$

A form of the **triangle inequality** — a side of a triangle cannot be longer than the sum of the other two sides. h_{SLD} is consistent.

- Every consistent heuristic is admissible (not vice versa) $\Rightarrow$ with consistency, **A^* is cost-optimal**.
- With a consistent h : the *first* time we reach a state, it's on an optimal path — we never re-add a state to the frontier or update *reached*.
- With an *inconsistent* h : we may end up with multiple frontier nodes for one state, costing time and space. (Felner *et al.*, 2011: worst effects rarely happen in practice.)

Search Contours

- A **contour** in the state space is like a contour on a topographic map: inside the $f=400$ contour, all nodes have $g(n) + h(n) \leq 400$.
- A* expands the frontier node of lowest f -cost $\Rightarrow$ it fans out from the start in **concentric bands of increasing f -cost**.
- Uniform-cost search also has contours, but of g -cost only — **circular**, spreading equally in all directions.
- A* with a *good* heuristic: the $g+h$ bands **stretch toward the goal**, becoming narrowly focused around an optimal path.

Monotonicity of g

Along any path, g -cost strictly increases (positive action costs) $\Rightarrow$ contour lines never cross.

The $f = g + h$ cost is monotonically *non-decreasing* along a path **iff** h is consistent.

Surely Expanded Nodes and Optimal Efficiency

Let C^* = cost of the optimal solution.

- A* expands **all** nodes reachable from the start on a path where every node has $f(n) < C^*$ — these are **surely expanded nodes**.
- A* may expand some nodes right on the “goal contour” ($f(n) = C^*$) before selecting a goal node.
- A* expands **no** nodes with $f(n) > C^*$.

Optimal efficiency

A* with a consistent heuristic is **optimally efficient**: any algorithm that extends search paths from the initial state, using the same heuristic information, must expand *all* the surely-expanded nodes (any one of them could be on the optimal path).

A* **prunes** away nodes not necessary for finding an optimal solution (e.g., in the Romania example, Timisoara and Zerind, children of the root, are never expanded once a $f=418$ solution is found).

The Catch: A^ Can Still Expand Exponentially Many Nodes*

- A^* is complete, cost-optimal, and optimally efficient — very satisfying! But it is **not the answer to all our searching needs**.
- For many problems, the number of nodes expanded can be **exponential in the length of the solution**.

Super-powered vacuum world

A vacuum that can clean any one square at cost 1 unit, in any order, without needing to visit it first. With N initially dirty squares, there are 2^N states (some subset cleaned). *All* of them lie on an optimal solution path, so $f(n) < C^*$ for all of them — **A^* would visit every one**.

⇒ We need **satisficing** (suboptimal but “good enough”) and **memory-bounded** variants.

Satisficing Search: Inadmissible Heuristics & Weighted A^*

- A^* expands a lot of nodes; we can explore fewer by accepting **suboptimal** (“good enough”, or **satisficing**) solutions.
- Using an **inadmissible heuristic** (may overestimate) risks missing the optimum, but can be more accurate and reduce nodes expanded.
- **Detour index**: multiplier on straight-line distance for typical road curvature (e.g., 1.3 means a 10-mile straight-line trip is really a 13-mile drive; typically 1.2–1.6).

Weighted A^* search

$$f(n) = g(n) + W \times h(n), \quad W > 1$$

Focuses the search toward a goal $\Rightarrow$ fewer states explored, faster search — but if the optimal path ever strays outside the weighted search’s contour, it won’t be found. Result costs somewhere between C^* and $W \times C^*$ (in practice, usually much closer to C^*).

A Spectrum of Evaluation Functions

Strategy	$f(n)$	Weight
A* search	$g(n) + h(n)$	$W = 1$
Uniform-cost search	$g(n)$	$W = 0$
Greedy best-first search	$h(n)$	$W = \infty$
Weighted A* search	$g(n) + W \times h(n)$	$1 < W < \infty$

You could call weighted A* “somewhat-greedy search”: like greedy best-first search it focuses toward a goal, but it won’t ignore path cost completely and will suspend a path making little progress at great cost.

More flavors of “good enough”

Bounded suboptimal search: solution guaranteed within factor W of optimal (weighted A* provides this). **Bounded-cost search:** cost $<$ some constant C . **Unbounded-cost search:** any cost, found quickly (e.g., **speedy search:** greedy best-first using estimated *number of actions* as h).

Memory-Bounded Search: The Problem With A^*

- A^* 's main issue is **memory**: a frontier state is stored *both* as a node in the frontier *and* as an entry in *reached*.
- Possible savings: use the **separation property** (Fig. 3.6) plus a no-U-turn rule so we only need to check the frontier, not all of *reached*; or keep **reference counts** and drop states from *reached* once no more ways remain to reach them.

Beam search

Limits the frontier to the best k nodes by f -score, discarding the rest. **Incomplete and suboptimal**, but fast and often finds good near-optimal solutions with limited memory. (Variant: keep every node within δ of the best f -score, so the kept set grows/shrinks adaptively.)

Iterative-Deepening A^* (IDA^*)

- IDA^* is to A^* what iterative deepening is to depth-first search: gives the benefits of A^* without keeping all reached states in memory, at the cost of revisiting some states.
- In standard ID, the cutoff is *depth*, incremented by 1 each iteration. In IDA^* , the cutoff is the *f-cost* ($g + h$): each iteration exhaustively searches an *f*-contour, then uses the smallest *f*-cost that exceeded the cutoff as the next contour.

The 8-puzzle

Each path's *f*-cost is an integer $\Rightarrow$ steady progress toward the goal each iteration; no more than C^* iterations (≤ 31 for the hardest instances).

For problems where every node has a *different* *f*-cost, each new contour might contain only one new node, and the number of iterations could equal the number of states — IDA^* can be inefficient there.

Recursive Best-First Search (RBFS): Idea

- Mimics standard best-first search using only **linear space**.
- Like a recursive DFS, but tracks f_limit = the f -value of the best *alternative* path from any ancestor. If the current node exceeds this limit, recursion unwinds to the alternative.
- As recursion unwinds, RBFS replaces each node's f -value with a **backed-up value** — the best f -value of its children — so it can decide later whether to re-expand a forgotten subtree.

Preview: how RBFS behaves on the Romania example (Fig. 3.23)

RBFS follows R. Vilcea, then “changes its mind” and tries Fagaras, then changes back — because every time the current best path is extended, its f -value tends to *increase* (h is usually less optimistic near the goal), so the second-best path can become the best path.

RBFS is optimal if h is admissible; space is linear in the depth of the deepest optimal solution, but time complexity is hard to characterize — it depends on heuristic accuracy and how often the best path changes.

Recursive Best-First Search (RBFS): Algorithm

```

1: function RBFS(problem, node, f_limit)
2:   if problem.Is-Goal(node.State) then return node
3:   end if
4:   successors  $\leftarrow$  List(Expand(node))
5:   if successors is empty then return failure,  $\infty$ 
6:   end if
7:   for each s in successors do s.f  $\leftarrow$  max(s.Path-Cost + h(s), node.f)
8:   end for
9:   while true do
10:    best  $\leftarrow$  the node in successors with lowest f-value
11:    if best.f > f_limit then return failure, best.f
12:    end if
13:    alternative  $\leftarrow$  second-lowest f-value among successors
14:    result, best.f  $\leftarrow$  RBFS(problem, best, min(f_limit, alternative))
15:    if result  $\neq$  failure then return result, best.f
16:    end if
17:  end while

```

MA and SMA*: Use All Available Memory*

- IDA* and RBFS suffer from using **too little** memory: between iterations IDA* retains only one number (the f -limit); RBFS retains more, but still only linear space.
- **MA*** (memory-bounded A*) and its simpler cousin **SMA*** let the algorithm use *all* the memory available.

SMA behavior*

Proceeds like A*, expanding the best leaf, until memory is full. Then, to add a new node, it **drops the worst leaf** (highest f -value), backing up that value to its parent (as in RBFS) — so an ancestor always knows the quality of the best path in a forgotten subtree, and regenerates it only when all other paths look worse.

- **Complete** if any reachable solution's depth d is less than the memory size; **optimal** if any optimal solution is reachable; else returns the best *reachable* solution.
- On very hard problems, SMA* can **thrash** — switching continually among candidate paths, like paging thrashing — making the problem intractable *from a time standpoint*, even though A* with unlimited memory would solve it easily.

Bidirectional Heuristic Search

- Can we combine bidirectional search with $f = g + h$? Unfortunately no guarantee of an optimal-cost or optimally efficient result, even with admissible h — efficiency proofs need to consider *pairs* of nodes (one per frontier), not individual nodes.
- Notation: $f_F(n) = g_F(n) + h_F(n)$ (forward), $f_B(n) = g_B(n) + h_B(n)$ (backward). A lower bound on a solution through nodes m (forward) and n (backward):

$$lb(m, n) = \max(g_F(m) + g_B(n), f_F(m), f_B(n))$$

Practical evaluation function

$$f_2(n) = \max(2g(n), g(n) + h(n))$$

Guarantees we never expand a node with $g(n) > C^*/2$ from either frontier — the two halves “meet in the middle.”

Front-to-end search: h_B estimates distance to the start. **Front-to-front** search: estimates distance to the *other frontier* directly (can be summarized, e.g., via a bounding box, rather than sampled point-by-point). Bidirectional search with f_2 and admissible h is complete and optimal; it helps most when h is only *average* quality.

The 8-Puzzle Revisited: Two Classic Heuristics

7	2	4
5		6
8	3	1

Start

	1	2
3	4	5
6	7	8

Goal

(shortest solution: 26 actions) $9!/2 = 181,400$ reachable states (15-puzzle: $16!/2 \approx 10$ trillion!)

- h_1 = number of **misplaced tiles** (blank excluded). For the state above, $h_1 = 8$. Admissible: every out-of-place tile needs ≥ 1 move.
- h_2 = sum of **Manhattan (city-block) distances** of tiles from goal positions — horizontal + vertical distance only (no diagonals). For the state above, $h_2 = 3+1+2+2+2+3+3+2 = 18$. Also admissible: every move brings at most one tile one step closer.

Neither overestimates the true cost (26).

Effective Branching Factor

Definition

If A^* generates N nodes total for a problem solved at depth d , the **effective branching factor** b^* is the branching factor a *uniform* tree of depth d would need to contain $N+1$ nodes:

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

E.g., if A^* finds a solution at depth 5 using 52 nodes, $b^* = 1.92$. b^* is usually fairly constant across nontrivial instances of a given domain — a well-designed heuristic has b^* close to 1.

Effective depth (Korf & Reid, 1998)

A better characterization: heuristic h reduces the **effective depth** by a constant k_h , giving total search cost $O(b^{d-k_h})$ vs. $O(b^d)$ uninformed. Accurate for Rubik's Cube and n -puzzle experiments (for solution lengths $> k_h$).

Comparing h_1 and h_2 Empirically

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
10	1033	116	48	1.85	1.43	1.27
14	6783	1683	364	1.77	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
22	175921	22955	2548	1.66	1.50	1.34
28	463234	202565	22055	1.53	1.49	1.36

(Averaged over 100 puzzles per depth;

abridged from Fig. 3.26.) h_2 is clearly better than h_1 , and both beat no heuristic (BFS) by orders of magnitude.

Domination

Because $h_2(n) \geq h_1(n)$ for all n , we say h_2 **dominates** h_1 . Domination $\Rightarrow$ efficiency: A^* using h_2 never expands more nodes than A^* using h_1 (with a consistent heuristic). **It's generally better to use a heuristic with higher values**, provided it's consistent and cheap to compute.

Generating Heuristics from Relaxed Problems

- h_1 and h_2 are each the *exact* solution cost for a **simplified (relaxed) version** of the 8-puzzle.
- A **relaxed problem** has fewer restrictions on actions; its state-space graph is a *supergraph* of the original (added edges = shortcuts).

Key fact

The cost of an optimal solution to a relaxed problem is an **admissible heuristic** for the original problem. Since it's an *exact* cost for the relaxed problem, it obeys the triangle inequality, so it is also **consistent**.

If the 8-puzzle action “A tile can move from X to Y if X is adjacent to Y **and** Y is blank” has conditions removed:

- | | |
|---|--|
| (a) move $X \rightarrow Y$ if X adjacent to Y | $\Rightarrow$ gives h_2 (Manhattan distance) |
| (b) move $X \rightarrow Y$ if Y is blank | $\Rightarrow$ a different heuristic |
| (c) move $X \rightarrow Y$ (unconditionally) | $\Rightarrow$ gives h_1 (misplaced tiles) |

Program Absolver (Prieditis, 1993) generates such heuristics automatically from formal problem definitions — and found the first useful heuristic for Rubik's Cube.

Combining Multiple Admissible Heuristics

If a collection $h_1, \dots, h_m$ is available and none dominates the others, take the **max**:

$$h(n) = \max\{h_1(n), \dots, h_m(n)\}$$

- Because each h_i is admissible, h is admissible (and consistent if all h_i are consistent).
- h **dominates** all of its components — best of all worlds!
- Drawback: $h(n)$ takes longer to compute. Alternatives: randomly pick one h_i per evaluation, or use ML to predict which will be most accurate (may yield an inconsistent h , but often faster in practice).

Generating Heuristics from Subproblems: Pattern Databases

*	2	4
*		*
*	3	1

Start

	1	2
3	4	*
*	*	*

Goal

A **subproblem**: get tiles 1, 2, 3, 4, and the blank into their correct positions (other tiles are “don’t care”).

- A **pattern database** stores the exact solution cost for *every* subproblem configuration ($9 \times 8 \times 7 \times 6 \times 5 = 15,120$ patterns here), built by searching *backward* from the goal (dynamic programming).
- Lookup gives an admissible heuristic h_{DB} , more accurate than Manhattan distance in some cases.
- Multiple databases (e.g., 1-2-3-4 and 5-6-7-8) can be **combined by taking the max**.

Disjoint Pattern Databases

Can we just add two pattern-database heuristics instead of maxing?

Not in general — solutions to the 1-2-3-4 and 5-6-7-8 subproblems will almost certainly share moves (unlikely 1-2-3-4 can be placed without ever touching 5-6-7-8), so summing double-counts.

The fix: disjoint pattern databases

Record only the *moves involving* 1-2-3-4 (not moves of other tiles) as the subproblem cost. Then the **sum** of the two disjoint databases is still a valid lower bound $\Rightarrow$ admissible and much more accurate.

- Combined heuristics can reduce nodes generated for random 15-puzzles by a factor of **1000**; disjoint databases solve them in a few **milliseconds**, a $10,000\times$ reduction over Manhattan distance.
- For 24-puzzles, speedups of roughly a factor of a **million** are possible.
- Works well for sliding-tile puzzles because each move affects only one tile $\Rightarrow$ the state naturally divides into independent subproblems.

Generating Heuristics with Landmarks

- Online map services find cost-optimal routes on graphs with tens of millions of vertices, in *milliseconds* — via **precomputation**.
- A perfect heuristic could be built by precomputing $C^*(v, u)$ for every pair of vertices, but that's $O(|V|^2)$ space and $O(|E|^3)$ time — practical only up to $\sim 10K$ vertices, not 10 million.

Landmark points

Choose a few (10–20) **landmark points** L . For each landmark and each vertex v , precompute and store $C^*(v, L)$ (exact optimal-path cost). Then:

$$h_L(n) = \min_{L \in \text{Landmarks}} C^*(n, L) + C^*(L, \text{goal})$$

If the optimal path passes through a landmark, h_L is exact; otherwise it **overestimates** — **inadmissible**. (If exact heuristics are available, once A^* reaches an optimal-path node, every subsequent expansion stays on that optimal path — $f = g + h$ stays constant at C^* .)

Shortcuts and the Differential Heuristic

Shortcuts

Some route-finders add **shortcuts** — artificial edges representing an optimal multi-action path. E.g., a precomputed shortcut between 100 major U.S. cities lets a Berkeley→NYU trip use the Sacramento↔Manhattan shortcut, covering 90% of the path in one action.

Differential heuristic (admissible and efficient)

$$h_{DH}(n) = \max_{L \in Landmarks} |C^*(n, L) - C^*(goal, L)|$$

Idea: take the exact cost from n to a landmark L beyond the goal, then subtract off the tail from $goal$ to L — giving the (approximate) cost from n to $goal$.

Best landmarks are spread around the **perimeter** of the graph (e.g., find the centroid, arrange k pie-shaped wedges around it, pick the farthest vertex in each wedge). Landmarks work especially well for roads because civil engineers build the fastest routes *between* landmarks.

Learning to Search Better: The Metalevel State Space

- Could an agent *learn* to search better, rather than following a fixed hand-coded strategy?
- **Metalevel state space:** each state captures the *internal (computational)* state of a program searching an ordinary (**object-level**) state space, e.g. A*'s current search tree.
- Each action in the metalevel space is a *computation step* that alters the internal state — e.g., each A* step expands a leaf and adds its successors.

Not every computation step helps

In the A* Romania trace, one step — expanding Fagaras — turned out unhelpful. For harder problems there can be *many* such missteps.

A **metalevel learning** algorithm can learn from these experiences to avoid exploring unpromising subtrees (Ch. 23), minimizing the **total cost** of problem solving: computation expense traded off against path cost.

Learning Heuristics from Experience

- Alternative to relaxed problems: **learn** h from experience — solve many 8-puzzles; each solution gives a (goal, path) example.
- A learning algorithm can approximate the true path cost for *other* states — but risks **inadmissibility** (an inevitable time/quality tradeoff, Ch. 19, Ch. 23).

Using features

ML works better with **features** of a state relevant to predicting h , not just the raw description. E.g. $x_1(n)$ = number of misplaced tiles, $x_2(n)$ = number of adjacent-in-state-but-not-in-goal tile pairs. Combine linearly:

$$h(n) = c_1 x_1(n) + c_2 x_2(n)$$

Constants c_1, c_2 fit to data from randomly generated configurations; both expected positive (more misplaced/incorrect pairs $\Rightarrow$ harder). Satisfies $h(n) = 0$ at goal states, but **not necessarily admissible or consistent**.

Summary (1/3): Problems and Uninformed Search

- Before an agent can search, a well-defined **problem** must be formulated: **initial state**, **actions**, **transition model**, **goal states**, **action cost function**.
- The environment is represented by a **state-space graph**; a **solution** is a path from initial state to a goal state.
- Search algorithms generally treat states/actions as **atomic** (features come in only for learning, Section 3.6.6).
- Algorithms are judged on **completeness**, **cost optimality**, **time complexity**, **space complexity**.

Uninformed methods (access only to the problem definition)

- **Best-first search** selects nodes via an evaluation function f .
- **Breadth-first**: shallowest first; complete, optimal for unit costs, exponential space.
- **Uniform-cost**: lowest $g(n)$ first; optimal for general costs.
- **Depth-first**: deepest first; neither complete nor optimal, but linear space. **Depth-limited** adds a bound.
- **Iterative deepening**: repeated depth-limited search with growing limits; complete (unit costs), BFS-like time, linear space.
- **Bidirectional**: two frontiers, stop when they meet.

Summary (2/3): Informed Search

- Informed methods use a **heuristic function** $h(n)$ estimating the cost of a solution from n ; may also use pattern databases, landmarks, etc.
- **Greedy best-first**: expands minimal $h(n)$; not optimal, often efficient.
- **A***: expands minimal $f(n) = g(n) + h(n)$; complete and optimal if h admissible. Space complexity remains a serious issue.
- **Bidirectional A***: sometimes more efficient than plain A*.
- **IDA***: iterative-deepening version of A*, addresses the space issue.
- **RBFS** and **SMA***: robust, optimal, limited-memory algorithms — given enough time, they solve problems A* runs out of memory on.
- **Beam search**: bounds frontier size — incomplete/suboptimal, but fast, often near-optimal.
- **Weighted A***: focuses search toward the goal, expanding fewer nodes at the cost of optimality.

The big picture

Tradeoffs always exist between **time**, **memory**, and **solution quality**. Domain knowledge (a good heuristic) is what buys efficiency.

Summary (3/3): Where Good Heuristics Come From

Performance of heuristic search depends critically on heuristic **quality**. We can construct good heuristics by:

- **Relaxing** the problem definition (dropping action preconditions) — the optimal cost in the relaxed problem is admissible & consistent for the original.
- Precomputing solution costs for **subproblems** in a **pattern database** (optionally **disjoint**, so costs can be summed rather than maxed).
- Defining **landmarks** and precomputing exact distances to/from them (differential heuristics stay admissible).
- **Learning** from experience, using features of the state to predict solution cost.

Key equation to remember

$$f(n) = g(n) + h(n)$$

Historical Notes (Selected)

- State-space search originated with Newell & Simon's Logic Theorist (1957) and GPS (1961); Bellman (1957) showed the importance of additive path costs; Nilsson (1971) put the field on solid theoretical footing.
- The 15-puzzle captivated the public in 1880; falsely credited to Sam Loyd, it was actually invented by Noyes Chapman in the mid-1870s. The general $n \times n$ version is NP-complete (Ratner & Warmuth, 1986).
- Rubik's Cube (1974, Ernő Rubik): Korf (1997) found optimal solutions via pattern databases + IDA*; Rokicki *et al.* (2014) proved every instance solvable in 26 moves (35 CPU-years of computation). Agostinelli *et al.* (2019) used reinforcement learning + Monte Carlo tree search for fast (if not always optimal) solving.
- A* itself, admissibility, and consistency trace to work summarized and unified across decades of AI and OR research (see the full chapter notes for details).